

17.9

GKE & EKS — Managed Kubernetes

Control plane managed, you own the nodes

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

What is Managed Kubernetes?

Running your own Kubernetes control plane (etcd, API server, scheduler) is complex — you must patch it, back it up, ensure HA. Managed K8s (GKE, EKS) means the cloud provider runs the control plane for you, often for free. You only manage worker nodes.

■ **Simple Analogy:** Self-managed K8s = owning a restaurant (build kitchen, hire chefs, manage health codes). Managed K8s = renting a fully equipped commercial kitchen — you just bring ingredients and cook.

GKE vs EKS — Side by Side

Feature	GKE (Google)	EKS (Amazon)
Control plane cost	Free (Standard) / \$0.10/hr (Enterprise)	\$0.10/hr per cluster
Node management	Standard (you manage) or Autopilot (Google-managed)	Managed Node Groups or Fargate
Networking	VPC-native pods (alias IPs)	AWS VPC CNI
Load Balancer	GCP GLB / NEG	AWS ALB / NLB
IAM integration	Workload Identity (GCP SA -> K8s SA)	IRSA (IAM Role for SA)
Upgrade strategy	1-click auto-upgrade	eksctl or managed console
Best for	GCP-native workloads	AWS-native workloads

GKE Autopilot — Fully Managed Nodes

GKE Autopilot removes node management entirely. You only deploy pods — Google provisions, patches, and scales nodes automatically. You pay per pod CPU/memory, not per node.

```
# Create GKE Autopilot cluster
gcloud container clusters create-auto my-cluster \
--region us-central1

# GKE Standard cluster (you manage nodes)
gcloud container clusters create my-cluster \
--num-nodes 3 \
--machine-type n2-standard-4 \
--region us-central1
```

EKS with IRSA — Secure AWS Access from Pods

IRSA (IAM Roles for Service Accounts) lets pods assume AWS IAM roles without storing credentials. A pod annotated with an IAM role gets temporary AWS credentials via a projected token — zero static secrets.

```
# Step 1: Create IAM Role with trust policy for the K8s service account
# Step 2: Annotate service account
apiVersion: v1
kind: ServiceAccount
metadata:
  name: s3-reader
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789:role/S3ReadRole

# Pod using this SA gets temporary AWS credentials automatically
# Java SDK picks up credentials via DefaultCredentialsProvider
S3Client s3 = S3Client.builder().region(Region.US_EAST_1).build();
```

Cluster Upgrade Best Practices

- Always upgrade control plane first, then node groups
- Test upgrades in a staging cluster first (same K8s version as prod)
- Use Surge Upgrades on EKS: launch new node, drain old node, terminate — zero disruption
- GKE auto-upgrades can be set to 'stable' channel — lags 3 versions behind latest
- Check API deprecations before upgrading (e.g., Ingress API changed in 1.22)

■ **Real-World Example:** Pinterest migrated from self-managed Kubernetes on EC2 to EKS, saving 40% of time their platform team spent on control plane operations. They use IRSA for all AWS service access — no IAM credentials stored anywhere in the cluster.